PROYECTO DE DATOS

Clima Hermosillo

De una API meteorológica a un dashboard interactivo

Python / PostgreSQL / Streamlit / Docker

01
Obtener

02
Almacenar

03
Visualizar

01 / PROPÓSITO

¿Qué resuelve el proyecto?

Un flujo completo de datos meteorológicos para una ubicación fija.

Consultar

Condiciones actuales estimadas para Hermosillo.

Temperatura, humedad, sensación térmica y viento.

Conservar

Historial local de capturas.

Versiones sucesivas de los pronósticos horarios.

Analizar

Temperatura y lluvia esperadas.

Diferencias frente al pronóstico de un día antes.

Alcance: consume modelos externos; no entrena un modelo propio.

02 / ARQUITECTURA

Tres servicios, un flujo de datos

Cada componente tiene una responsabilidad independiente.

Open-Meteo
API externa

>

collector
Extracción en Python

>

PostgreSQL
Almacenamiento

>

Streamlit
Visualización

HTTP + JSON

Inserciones SQL

Consultas SQL

Navegador :8501

Docker Compose coordina collector, db y dashboard dentro de una red interna.

03 / INGESTA

Cómo funciona el recolector

collector/worker.py | Ubicación: 29.0729, -110.9559 | America/Hermosillo

1. Consultar

Obtiene condiciones actuales y pronóstico de tres días.

Consulta también predicciones con un día de anticipación.

2. Transformar

Convierte fechas y arreglos JSON en filas.

Alinea por hora los datos para la comparación.

3. Guardar

Inserta en tres tablas dentro de una transacción.

Espera 600 segundos al finalizar y vuelve a comenzar.

Requests: HTTP y JSON. Psycopg: conexión y escritura en PostgreSQL.

04 / PERSISTENCIA

Tres tablas con funciones distintas

schema.sql | Identificadores, índices temporales y conservación de versiones.

observations

Instantánea de condiciones actuales.

fetched_at: captura.
observed_at: hora del dato.

forecasts

Una fila por hora de cada versión descargada.

fetched_at + forecast_for identifica una fila única.

comparisons

Valores actuales y predicciones anteriores.

valid_hour permite agrupar las comparaciones por hora.

Las tablas no tienen claves foráneas; se consultan como series temporales.

05 / ANÁLISIS

¿Qué significa la desviación?

La interfaz calcula: valor actual estimado menos predicción anterior.

Ejemplo

Actual estimado: 34 °C.

Pronóstico anterior: 32 °C.

Desviación: +2 °C.

Interpretación

Positiva: mayor que lo previsto.

Negativa: menor que lo previsto.

Cero: ambos coinciden.

Alcance

Compara estimaciones de modelos.

La lluvia se compara en intervalos horarios.

No mide error contra una estación.

Para evaluar precisión real se necesitarían mediciones independientes de referencia.

06 / INTERFAZ

Un dashboard que consulta la base

app/dashboard.py | Streamlit + Pandas + Plotly + SQLAlchemy.

Vista

Métricas actuales.

Pronóstico de temperatura, lluvia y probabilidad.

Historial y desviaciones.

Consultas

Última observación.

Última versión de pronóstico.

Hasta 300 capturas históricas y comparaciones.

Actualización

Fragmento cada 60 segundos con sesión activa.

Caché de resultados: 50 segundos.

Nuevas capturas: ~10 minutos.

La interfaz puede funcionar con los datos almacenados aunque el recolector falle.

Configuración de Docker Compose

compose.yaml define imágenes, variables, red, volumen y dependencias.

db

postgres:16-alpine.

Volumen postgres_data para persistencia.

schema.sql inicializa una base vacía.

collector

Imagen python:3.12-slim.

DATABASE_URL conecta a db:5432.

UPDATE_SECONDS = 600.

dashboard

Imagen python:3.12-slim.

Publica el puerto 8501.

Streamlit escucha en 0.0.0.0.

Las dependencias Python tienen versiones fijadas en los archivos requirements.txt.

08 / OPERACIÓN

Arranque, red y recuperación

Los servicios internos utilizan el nombre db para encontrar PostgreSQL.

Arranque

pg_isready comprueba la disponibilidad de la base.

collector y dashboard esperan ese estado saludable.

Fallos

HTTP con timeout de 30 segundos.

Excepciones registradas en logs y reintento posterior.

Escrituras con rollback si fallan.

Persistencia

down conserva el volumen.

down -v elimina los datos.

collector y dashboard: restart unless-stopped.

Credenciales de demostración en Compose: deben gestionarse como secretos en producción.

09 / EVALUACIÓN

Fortalezas y límites del proyecto

Aspectos que conviene explicar con precisión durante la exposición.

Fortalezas

Separación de funciones.

Historial y versiones persistentes.

Consultas SQL parametrizadas.

Despliegue con Compose.

Límites

Tres días no garantizan 72 horas futuras.

Temperatura actual y hora redondeada no coinciden siempre.

Sin política de limpieza.

Mejoras

Alinear instantes e intervalos.

Separar fallos de las dos APIs.

Gestionar secretos y migraciones.

Agregar monitoreo y retención.

Si falla la API de predicciones anteriores, el ciclo actual tampoco almacena observaciones.

10 / DEMOSTRACIÓN

Cómo presentar el sistema en vivo

Ejecutar desde la carpeta clima-hermosillo. Iniciarlo con anticipación.

1 Iniciar	docker compose up --build -d
2 Revisar servicios	docker compose ps
3 Ver actividad	docker compose logs --tail=30 collector
4 Abrir la interfaz	http://localhost:8501

Mostrar: métricas > pronóstico > desviaciones > historial almacenado.

11 / CIERRE

De datos externos a información útil

El aporte central es integrar y conservar un flujo de datos reproducible.

Obtener

Python automatiza la consulta de una fuente meteorológica.

No hay entrenamiento de un modelo propio.

Conservar

PostgreSQL guarda capturas y versiones de pronósticos.

Permite analizar su evolución en el tiempo.

Comunicar

Streamlit convierte registros en métricas y gráficas.

Docker Compose reúne los componentes.

Base del análisis: código local. Compose y sintaxis validados; ejecución no verificada.